

1. Introdução à multiplexação em robótica

No desenvolvimento de um robô seguidor de linha, o uso de vários sensores ópticos é fundamental para que o robô consiga identificar a pista com precisão e fazer curvas suaves. Porém, microcontroladores comuns possuem um número muito limitado de entradas analógicas (apenas 6 pinos, de A0 a A5). Se o projeto precisar de muitos sensores para fazer uma leitura da linha, os pinos do Arduino sozinhos não serão suficientes. É para resolver esse problema de falta de pinos que usamos a multiplexação analógica.

1.1 O que é um multiplexador?

Um multiplexador funciona como uma chave seletora eletrônica. Ele possui várias portas de entrada de sinal e apenas uma porta de saída comum.

- **Funcionamento:** Ele recebe o sinal de muitos canais independentes ao mesmo tempo, mas conecta apenas um canal por vez à sua saída comum. É uma lógica de "muitos para um".

No caso do seguidor de linha, nós conectamos todos os sensores nas entradas do multiplexador, e ligamos a única saída comum dele em apenas uma porta analógica do Arduino. O Arduino então comanda o chip para alternar rapidamente entre os sensores, lendo um por um.

1.2 O que é um demultiplexador?

O demultiplexador faz o caminho inverso do multiplexador.

- **Funcionamento:** Ele recebe o sinal de uma única linha comum e escolhe para qual das saídas disponíveis esse sinal será enviado. É uma lógica de "um para muitos". Um exemplo prático seria usar um único pino do Arduino para acender vários LEDs diferentes, um por vez.

Nota: No nosso robô, o processo acontece em duas etapas. Primeiro, o chip escolhido (CD74HC4067) faz a multiplexação física, unindo os sinais de todos os sensores em um único pino analógico do arduino. Logo em seguida, o código do Arduino faz a demultiplexação, separando essa leitura única e distribuindo os valores corretos para cada um dos 8 sensores na memória.

2. Análise técnica do circuito integrado CD74HC4067

O CD74HC4067 é o chip que usaremos como chave seletora eletrônica. Ele possui 16 canais independentes e 1 pino comum.

As três características do datasheet que precisamos dominar para o projeto são:

- **Bidirecional:** O sinal pode fluir para os dois lados. No robô, usaremos o sentido que leva o sinal dos sensores para o pino analógico do Arduino.
- **Troca rápida:** O chip desliga um canal antes de ligar o próximo (em cerca de 6 nanossegundos). Isso impede que as leituras dos sensores se misturem.
- **Pino enable (E):** Funciona com lógica invertida. Ele precisa ficar conectado no GND (0) para o chip permanecer ligado. Se receber 5V (1), o chip desliga.

3. Lógica dos pinos de endereço para isolar 8 canais

Como vimos, o CD74HC4067 possui 16 canais (de 0 a 15). Para escolher qual canal vai ser lido, o chip utiliza 4 pinos de endereço: S0, S1, S2 e S3. O Arduino precisa enviar sinais binários (0 ou 1) para esses pinos para abrir o canal correto ($2^4 = 16$ combinações).

Na linguagem binária, cada pino tem um "peso" matemático, lido da direita para a esquerda:

- **S0** (bit menos significativo): vale 1;
- **S1**: vale 2;
- **S2**: vale 4;
- **S3** (bit mais significativo): vale 8.

Para abrir o canal 5, por exemplo, o arduino ativa S2 (que vale 4) e S0 (que vale 1), pois $4 + 1 = 5$. No nosso seguidor de linha, precisamos ler apenas 8 sensores (canais de 0 a 7). Se usássemos os 4 pinos de controle conectados ao arduino, gastaríamos pinos da placa sem necessidade. Olhando para a tabela verdade do datasheet do chip, podemos ver o comportamento dos pinos para os primeiros 8 canais:

Canal	E ⁻ (Enable)	S3	S2	S1	S0
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	0	1	1
4	0	0	1	0	0
5	0	0	1	0	1
6	0	0	1	1	0
7	0	0	1	1	1

Repara na coluna do pino S3: para qualquer canal entre 0 e 7, o valor do pino S3 é sempre 0 (nível lógico BAIXO). Se o S3 fosse ativado (nível 1), ele imediatamente somaria "8" à conta matemática, fazendo o chip pular para os canais de 8 a 15. Ao mantê-lo em 0, a soma máxima possível dos pinos restantes (S2, S1 e S0) é exatamente $4 + 2 + 1 = 7$.

3.1 Como isolar os 8 canais na prática

Sabendo que o S3 nunca precisa ser ativado para lermos nossos 8 sensores:

1. **Ligue o pino S3 diretamente no GND:** ao fazer isso, você trava o bit S3 em 0 permanentemente. O chip fica fisicamente impedido de somar o "peso 8", isolando o componente para funcionar apenas nos canais de 0 a 7.
2. **Ligue o pino enable (E) no GND:** como vimos na seção 2, isso envia o sinal 0 para manter as chaves do chip sempre ativas.
3. **Conecte apenas S0, S1 e S2 ao arduino:** agora, você precisa de apenas 3 pinos digitais para controlar as leituras. Com 3 pinos, fazemos as combinações binárias de 000 a 111, que cobrem exatamente as 8 possibilidades que precisamos ($2^3 = 8$).

Ao travar o S3 no GND, transformamos o nosso chip de 16 canais em um chip de 8 canais. Isso economiza pinos do arduino e nos permite usar exatamente a mesma lógica de código de um multiplexador menor.

4. Estudo da lógica do código (código anexado pelo Bozelli)

No nosso robô, o chip de hardware faz a multiplexação: ele junta os 8 sensores em um único pino físico, o A0. Agora, o código precisa realizar uma varredura rápida: ler esse pino A0 consecutivamente, mudando os canais do chip no tempo, e organizar cada leitura na sua própria variável dentro da memória.

4.1 Organização dos dados

Em vez de criar 8 variáveis soltas (como `int sensor1`, `sensor2...`), que seriam difíceis de gerenciar, agrupamos todas as leituras em uma estrutura de dados única (`struct`) que contém um array:

```
struct dados_sensores {  
    int sensor[8];  
};
```

Isso permite o uso de loops (`for`) no programa. O número do canal do chip (0 a 7) passa a ser exatamente o índice da posição onde o valor será guardado na memória (`sensor[0]` até `sensor[7]`).

4.2 A função *lerMux()* (varredura e distribuição de sinais)

Esta função gerencia a leitura do chip multiplexador. Ela executa um loop de 0 a 7. Em cada passo, ela calcula o endereço binário do canal, envia os sinais de controle para os pinos S0, S1 e S2, e salva o valor do pino analógico comum na posição correta do array.

```
dados_sensores lerMux() {  
    dados_sensores dadosColetados;  
  
    for (int canal = 0; canal < 8; canal++) {  
        // Lógica binária para definir os pinos S0, S1 e S2  
        digitalWrite(pinosEndereco[0], (canal & 0x01)); // Bit 0 (S0)  
        digitalWrite(pinosEndereco[1], (canal & 0x02) >> 1); // Bit 1 (S1)  
        digitalWrite(pinosEndereco[2], (canal & 0x04) >> 2); // Bit 2 (S2)  
  
        // Lê o canal atual sempre pelo mesmo pino SIG  
        dadosColetados.sensor[canal] = analogRead(pinoSIG);  
    }  
  
    return dadosColetados;  
}
```

Como funciona a matemática dos pinos (& e >>):

O código usa operações de bits (chamadas de máscaras binárias) para descobrir quais pinos devem ser ligados ou desligados para formar o número do canal atual.

- O operador & é uma "máscara": faz uma comparação lógica bit a bit. A regra dele é simples: o resultado só será 1 se os dois bits comparados forem 1. Se houver algum zero, o resultado é 0. No código, usamos máscaras como 0x01 (que significa o número 1 em binário: 001) e 0x02 (que é o número 2 em binário: 010) para isolar o bit que queremos analisar.
- O Operador >> é um "empurrão" para a direita: este operador serve para deslocar os bits para a direita, como se os estivéssemos a empurrar. Se tiver o binário 10 e fizer >> 1, o 0 cai para fora e o 1 passa para a casa da direita, transformando-se em 1 (ou 01).

Por exemplo, quando o loop chega no canal 5 (que em binário é escrito como 101):

- $(5 \& 0x01)$ isola o primeiro bit (à direita) → O pino S0 recebe 1 (HIGH).
- $(5 \& 0x02) \gg 1$ isola o segundo bit (o do meio) → O pino S1 recebe 0 (LOW).
- $(5 \& 0x04) \gg 2$ isola o terceiro bit (à esquerda) → O pino S2 recebe 1 (HIGH).

A combinação 101 nos pinos de endereço faz o chip conectar fisicamente o sensor número 5 ao pino A0. A linha *analogRead(pinoSIG)* faz a leitura e a armazena na gaveta *sensor[5]*. O loop repete esse processo para todos os 8 canais em poucos microssegundos.

4.3 Normalização dos dados

Os sensores analógicos geram valores brutos que variam de 0 a 1023. Para facilitar os cálculos matemáticos que definem o controle de curvas dos motores, o código passa esses dados por uma função de tratamento que usa o comando *map()*:

```
dados_sensores normalizacao(dados_sensores dados) {  
    for (int i = 0; i < 8; i++) {  
        dados.sensor[i] = map(dados.sensor[i], 0, 1023, minVal, maxVal);  
    }  
    return dados;  
}
```

O *map()* pega o valor bruto (0-1023) e o converte proporcionalmente para uma escala menor e mais simples de processar, configurada para ir de 0 a 255.

4.4 O fluxo principal (loop)

Com as funções bem divididas, o ciclo de funcionamento do robô fica simples:

1. *dadosBrutos = lerMux();* → Realiza a varredura física do chip e distribui os dados em 8 posições separadas na memória.
2. *dadosNormalizados = normalizacao(dadosBrutos);* → Ajusta a escala de todos os valores de 0 a 255.
3. Os dados tratados ficam prontos para uso em algoritmos de direção (como o controle PID do seguidor de linha).